



SecureDapp

DApp Developers and Smart Contract Auditors

SMART CONTRACT SECURITY AUDIT of CRYPT WALLET CONTRACTS



Smart Contract Audit of CrypWallet

NOVEMBER 11th, 2025 | v. 1.1



TABLE OF CONTENTS

AUDIT INTRODUCTION	3
--------------------	---

AUDIT DOCUMENT	4
----------------	---

AUDIT SCOPE	4
• Initial Review Scope	

AUDIT SUMMARY	8
---------------	---

AUDIT METHODOLOGY	9
-------------------	---

SYSTEM OVERVIEW	13
-----------------	----

FINDINGS	14
----------	----

STATIC ANALYSIS REPORT	15
------------------------	----

MANUAL REVIEW	18
---------------	----

UNIT TEST REPORT	21
------------------	----

DISCLAIMER	24
------------	----

ABOUT SECUREDAPP	27
------------------	----



AUDIT INTRODUCTION

Auditing Firm	SecureDApp Auditors
Audit Architecture	SecureDApp Auditing Standard
Language	Solidity
Client Firm	CRYP WALLET
Youtube	-
Twitter	-
Linkedin	-
Facebook	-
Instagram	-
Report Date	November 11th, 2025



AUDIT DOCUMENT

Name	Smart Contract Code Review and Security Analysis Report for CrypWallet
Approved By	Himanshu Gautam CTO at SecureDApp
Type	Custom
Platform	EVM / TVM
Language	Solidity
Changelog	11.11.2025 – Final Review

AUDIT SCOPE

The scope of this report is to audit the smart contract source code of CrypWallet contracts.

Our client provided us with multiple smart contracts contained in the repository at:

- <https://github.com/FABC-LABS/crypwallet>

The contract is written in Solidity. After initial research, we agreed to perform the following tests and analyses as part of our well-rounded audit:

- Smart contract behavioural consistency analysis
- Test coverage analysis
- Penetration testing: checking against our database of vulnerabilities and simulating manual attacks against the contracts
- Static analysis
- Manual code review and evaluation of code quality
- Analysis of GAS usage
- Contract analysis with regards to the host network



Initial Review Scope

Contract Link	https://github.com/FABC-LABS/crypwallet
Commit Hash	bd49083cdfd82b936ed852444af81b70f61a65ea
Functional Requirements	Partial documentation provided. README.md
Technical Requirements	Partial documentation provided. README.md
Contracts Addresses	N/A
Contracts	Swapper.sol, TransferHandler.sol, AdminController.sol

Final Review Scope

Contract Link	https://github.com/FABC-LABS/crypwallet
Commit Hash	7b1654f1a80f798df0829a497ccf1dad3e0e67c7
Functional Requirements	Partial documentation provided. README.md
Technical Requirements	Partial documentation provided. README.md
Contracts Addresses	N/A
Contracts	Swapper.sol, TransferHandler.sol, AdminController.sol



Severity Definitions

Risk Level	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to asset loss or data manipulation.
High	High-level vulnerabilities are difficult to exploit; however, they also have a significant impact on smart contract execution, e.g., public access to crucial functions.
Medium	Medium-level vulnerabilities are important to fix; however, they cannot lead to asset loss or data manipulation.
Low	Low-level vulnerabilities are mostly related to outdated, unused, etc. code snippets that cannot have a significant impact on execution.
Informational	Issue listed to improve understanding, readability and quality of code

All statuses which are identified in the audit report are categorized here for the reader to review:

Status Type	Definition
Open	Risks are open.
Acknowledged	Risks are acknowledged, but not fixed.
Resolved	Risks are acknowledged and fixed.



AUDIT SUMMARY

The SecureDApp team has performed a line-by-line manual analysis and automated review of smart contracts. Smart contracts were analyzed mainly for common contract vulnerabilities, exploits, and manipulation hacks. According to the audit:

Status	Critical	High	Medium	Low	Informative
Open	0	0	0	0	0
Acknowledged	0	0	0	0	0
Resolved	6	0	1	3	0



AUDIT METHODOLOGY

SecureDApp scans contracts and reviews codes for common vulnerabilities, exploits, hacks and backdoors.

Mentioned are the steps used by SecureDApp to audit smart contracts:

- a. Smart contract source code review:
 - i. Review of the specifications, sources, and instructions provided to SecureDApp to make sure we understand the audit scope, intended business behaviour, overall architecture, and the project's goal.
 - ii. Manual review of code which is the process of reading source code line-by-line to identify potential vulnerabilities.
- b. Test coverage analysis: (Unit testing)
 - i. Test coverage analysis is the process of determining whether the test cases are covering the code and how much code is exercised when we run those test cases.
- c. Static analysis:
 - i. Run a suite of vulnerability detectors to find security concerns in smart contracts with different impact levels.
- d. Symbolically executed tests: (SMTChecker testing) (Taint analysis)
 - i. Symbolic execution is analysing a program to determine what inputs cause each part of a program to execute.
 - ii. Check for security vulnerabilities using static and dynamic analysis
- e. Property-based analysis (Fuzz tests)(Invariant testing)
 - i. Run the execution flow multiple times by generating random sequences of calls to the contract.
 - ii. Asserts that all the invariants hold true for all scenarios.
- f. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
- g. Specific, itemised, actionable recommendations to help you take steps to secure your smart contracts.

Automated 5S frameworks are used to assess the smart contract vulnerabilities

- Consensys Tools
- SWC Registry
- Solidity Coverage
- Open Zeppelin Code Analyser
- Solidity Shield Scan



We have audited the smart contracts for commonly known and more specific vulnerabilities. Below is the list of smart contract tests, vulnerabilities, exploits, and hacks:

ID	Description	Status
EEA 3.3	Oracle Manipulation	N/A
EEA 3.3	Bad Randomness - VRF	N/A
S60	Assembly Usage	Passed
S59	Dangerous usage of block.timestamp	Passed
EEA 3.7	Front-Running Attacks	N/A
EEA 3.7	Back-Running Attacks	N/A
EEA 3.7	Sandwich Attacks	N/A
DASP	Gas Griefing Attacks	Passed
DASP	Force Feeding	Passed
SCSVS V2	Access Control	Passed
DASP	Short Address Attack	Passed
DASP	Checks Effects Interactions	Passed
EEA 4.1	No Self-destruct	Passed
SCSVS V14	Decentralized Finance Checks	Passed



Slither Tests	Checks for ERC's conformance	Passed
Coverage	Unit tests with 100% coverage	Failed
Gas Reporter	Gas usage & limitations	Passed
Echidna Tests	Malicious input handling	Passed
SWC-101	Integer Overflow and Underflow	Passed
SWC-102	Outdated Compiler Version	Passed
SWC-103	Floating Pragma	Passed
SWC-104	Unchecked Call Return Value	Passed
SWC-105	Unprotected Ether Withdrawal	Passed
SWC-106	Unprotected SELF-DESTRUCT Instruction	Passed
SWC-107	Re-entrancy	Passed
SWC-108	State Variable Default Visibility	Passed
SWC-109	Uninitialized Storage Pointer	Passed
SWC-110	Assert Violation	Passed
SWC-111	Use of Deprecated Solidity Functions	Passed
SWC-112	Delegate Call to Untrusted Callee	Passed



SWC-113	DoS with Failed Call	Passed
SWC-114	Transaction Order Dependence	Passed
SWC-115	Authorization through tx.origin	Passed
SWC-116	Block values as a proxy for time	Passed
SWC-117	Signature Malleability	Passed
SWC-134	Message call with the hardcoded gas amount	Passed
SWC-135	Code With No Effects (Irrelevant/Dead Code)	Informational
SWC-136/SCSVS V3	Unencrypted Private Data On-Chain	Passed



SYSTEM OVERVIEW

Privileged roles

1. Contract Admin Role, Signer Role and Owner Role
 - a. Set through the constructor

Risks

1. The impact of the owner role being compromised would have a huge impact on the protocol.
2. Centralisation risk is the most common cause of cryptography asset loss.
3. Compromising the Admin Role may lead to all users' asset loss.



FINDINGS

Centralization Risk

Centralization risk is the most common cause of dapp's hacks. When a smart contract has an active contract ownership, the risk related to centralization is elevated. There are some well-intended reasons to be an active contract owner, such as:

- Contract owners can be granted the power to `pause()` or `lock()` the contract in case of an external attack.
- Contract owners can use functions like, `include()`, and `exclude()` to add or remove wallets from fees, swap checks, and transaction limits. This is useful to run a presale, and to list on an exchange.

Authorizing a full centralized power to a single body can be dangerous. Unfortunately, centralization related risks are higher than common smart contract vulnerabilities. Centralization of ownership creates a risk of rug pull scams, where owners cash out tokens in such quantities that they become valueless. The most important question to ask here is, how to mitigate centralization risk? Here's SecureDApp's recommendation to lower the risks related to centralization hacks:

- Smart contract owner's private key must be carefully secured to avoid any potential hack.
- Smart contract ownership should be shared by multi-signature (multi-sig) wallets.
- Smart contract ownership can be locked in a contract, user voting, or community DAO can be introduced to unlock the ownership.

CrypWallet Centralization Status

- Cryp Wallet smart contract has Contract Admin, Owner & Sign Authority Role.
- To avoid centralisation risk, consider using Openzeppelin Access Control instead of Ownable.



MANUAL REVIEW

Identifier	Definition	Severity
CEN-01	Incorrect Logic in transfer() Function	Critical

Contract: AdminController.sol

The AdminController.transfer() function uses incorrect conditional logic when validating token amount and balance checks, which can lead to unintended reverts or missed invalid transfers.

- `if (amount == 0 && IERC20(token).balanceOf(address(this)) <= amount) {`

Here, the condition uses the logical && (AND) operator.

- This means the function only reverts if both conditions are true simultaneously (amount is zero and balance $\leq$ amount).
- However, the correct logic should revert if either of these is true — amount being zero or insufficient balance.

Additionally, the comparison should use `<` instead of `<=` because if the contract's balance exactly equals the transfer amount, the transfer should still succeed.

Impact:

- Allows zero-value transfers that waste gas and emit misleading success.
- May revert valid transfers if the contract holds exactly the amount of tokens.
- Potential denial of service if downstream logic relies on accurate validation.

RECOMMENDATION

Use :: `if (amount == 0 || IERC20(token).balanceOf(address(this)) < amount)) {`

Status: Resolved



Identifier	Definition	Severity
CEN-02	Helper Contract Function Calls Will Fail	Critical

Contract: AdminController.sol

The functions that modify settings of TransferHandler and SwapHandler will fail at runtime because these external contracts likely restrict such calls to their owner only, while here they are being called by CrypwalletFeeHandler

Impact:

- AdminController Administrative functions will fail permanently after deployment unless ownership is manually reassigned to this handler.

RECOMMENDATION

Modify the helper contracts to grant `AdminController` a privileged role (e.g. `onlyController`).

Status: Resolved



Identifier	Definition	Severity
CEN-03	Global adminFeePercentage and adminGasFee for All Tokens	Critical

Contract: TransferHandler.sol and Swapper.sol

A single adminFeePercentage value is used for all token types in transferToken, making it impossible to set different fee rates per token. Also adminGasFee = 5; // Set globally for all tokens

This design doesn't account for tokens with different economic models or decimal systems. If tokens require distinct fees, this approach causes operational risk and incorrect deductions.

Impact:

- Incorrect or unfair fee application across various tokens.
- Potential financial miscalculation or loss to users/admin.

RECOMMENDATION

Use a mapping for per-token fees:

```
mapping(address => uint256) public tokenFee;
```

Status: Resolved



Identifier	Definition	Severity
CEN-04	No Access Control in transferEth()	Critical

Contract: TransferHandler.sol

The transferEth() function lacks any ownership or signature validation, allowing anyone to trigger Ether transfers to arbitrary addresses.

Any external caller can invoke this and supply recipients. If the contract holds ETH (via receive/fallback), it can be **fully drained**.

Impact:

- Unrestricted access can lead to total loss of ETH balance.
- Attackers can repeatedly call the function to empty funds.

RECOMMENDATION

Restrict access — either require ownership or signature verification:

Status: Resolved



Identifier	Definition	Severity
CEN-05	Missing Validation for Received ETH Amount	Critical

Contract: TransferHandler.sol

The function doesn't validate whether `msg.value` matches the total intended transfer amount plus fee.

Details:

Inside `transferEth`, only per-recipient and per-amount checks are made, but no total comparison against `msg.value`.

Impact:

- Mismatch between sent ETH and distributed amount can lead to partial or failed transfers.
- Possible underpayment exploits or unhandled leftovers.

RECOMMENDATION

Add total value validation:

```
require(msg.value == totalAmount + fee, "Incorrect ETH sent");
```

Status: Resolved



Identifier	Definition	Severity
CEN-06	Proper unit test cases with > 95% coverage	Critical

Unit Test Cases

- **Issues:**
 - Missing proper unit tests to include all positive and negative scenarios to validate business logic and failure cases

RECOMMENDATION:

Need well-structured unit test cases that thoroughly validate contract logic and behavior. Test coverage exceeds 95%, indicating a strong commitment to quality assurance and functional correctness.

Status: Resolved



Identifier	Definition	Severity
MED-01	Unlimited Token Approval	MEDIUM

Contract: Swapper.sol

The contract grants `type(uint256).max` allowance to the DEX address, posing a risk if that DEX or its integration is compromised.

Impact:

- Potential token theft if the DEX contract is malicious or upgraded.
- Loss of user or admin tokens without further interaction.

RECOMMENDATION

Limit allowance to the required amount:

```
SafeERC20.safeIncreaseAllowance(IERC20(tokenIn), DexAddresses[0], amount);
```

Status: Resolved



Identifier	Definition	Severity
LOW-01	Redundant Split Between fee and excludedFee	Low

Contract: TransferHandler.sol

`transferToken()` performs two separate transfers for `fee` and `excludedFee`, increasing gas use.

Details:

Each fee type (`adminFee` and `gasFee`) triggers a separate `safeTransferFrom`, even though both go to the same wallet.

Impact:

- Unnecessary double ERC20 calls, higher gas, and potential for one to succeed while other fails.

RECOMMENDATION

Combine them before transferring:

```
uint256 totalFee = fee + gasFee;
if (totalFee > 0) {
    SafeERC20.safeTransferFrom(token, _from, adminWallet, totalFee);
}
```

Status: Resolved



Identifier	Definition	Severity
LOW-02	nonReentrant Used Irrelevantly	Low

Contract: AdminController.sol

The nonReentrant modifier is applied to several functions that do not perform external calls or send Ether/tokens, making it redundant.

- For example: `function setAdminFeeForTransfer(uint256 _newFee) external onlyRole(ADMIN_ROLE) nonReentrant`

These setter functions simply call controlled helper methods or update internal state; they do not transfer funds or involve external call chains that could trigger reentrancy.

Impact:

- Unnecessary gas overhead.
- Code bloat and reduced readability.

RECOMMENDATION

Remove `nonReentrant` from functions that don't modify balances or send external transfers. Keep it only for:

- `transferOwnership`
- `withdraw`
- `withdrawETH`
- `changeSwapAndtransferHandler`

Status: Resolved



Identifier	Definition	Severity
LOW-03	Missing Zero Address Checks	Low

Contract: AdminController.sol

The constructor lacks validation for admin, _thandler, and _sHandler addresses, allowing deployment with zero addresses.

The transferOwnership() function allows transferring ownership to the zero address, which would lock the contract's administrative privileges permanently.

RECOMMENDATION:

Add explicit zero address checks:

```
if (admin == address(0) || address(_thandler) == address(0) || address(_sHandler) == address(0)) {  
    revert INVALIDADDRESS(admin);  
}
```

Status: Resolved



DISCLAIMER

SecureDApp Auditors provides an easy-to-understand audit of solidity source code (commonly known as smart contracts).

The smart contract for this particular audit was analyzed for common contract vulnerabilities and centralisation exploits. This audit report makes no statements or warranties on the security of the code. This audit report does not provide any warranty or guarantee regarding the absolute bug-free nature of the smart contract analysed, nor do it provide any indication of the client's business, business model or legal compliance. This audit report does not extend to the compiler layer, any other areas beyond the programming language, or other programming aspects that could present security risks. Cryptographic tokens are emergent technologies; they carry high levels of technical risks and uncertainty. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. This audit report could include false positives, false negatives, and other unpredictable results.

CONFIDENTIALITY

This report is subject to the terms and conditions (including, without limitations, description of services, confidentiality, disclaimer and limitation of liability) outlined in the scope of the audit provided to the client. This report should not be transmitted, disclosed, referred to, or relied upon by any individual for any purpose without SecureDApp's prior written consent.

NO FINANCIAL ADVICE

This audit report does not indicate the endorsement of any particular project or team, nor guarantees its security. No third party should rely on the reports in any way, including to make any decisions to buy or sell a product, service or any other asset. The information provided in this report does not constitute investment advice, financial advice, trading advice, or any other sort of advice and you should not treat any of the report's content as such. This audit report should not be used in any way to make decisions around investment or involvement. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. For avoidance of doubt, services, including any associated audit reports or materials, shall not be considered or relied upon as any form of financial, tax, legal, regulatory, or other advice.

TECHNICAL DISCLAIMER



All services, audit reports, smart contract audits, other materials, or any products or results of the use thereof are provided “as is” and “as available” and with all faults and defects without warranty of any kind. To the maximum extent permitted under applicable law, SecureDApp hereby disclaims all warranties, whether expressed, implied, statutory, or otherwise with respect to services, audit report, or other materials. Without limiting the foregoing, SecureDApp specifically disclaims all implied warranties of merchantability, fitness for a particular purpose, title and non-infringement, and all warranties arising from the course of dealing, usage, or trade practice. Without limiting the foregoing, SecureDApp makes no warranty of any kind that all services, audit reports, smart contract audits, or other materials, or any products or results of the use thereof, will meet the client’s or any other individual’s requirements, achieve any intended result, be compatible or work with any software, system, or other services, or be secure, accurate, complete, free of harmful code, or error-free.

TIMELINESS OF CONTENT

The content contained in this audit report is subject to change without any prior notice. SecureDApp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you access using the internet or other means, and assumes no obligation to update any information following the publication.

LINKS TO OTHER WEBSITES

This audit report provides, through hypertext or other computer links, access to websites and social accounts operated by individuals other than SecureDApp. Such hyperlinks are provided for your reference and convenience only and are the exclusive responsibility of such websites’ and social accounts’ owners. You agree that SecureDApp is not responsible for the content or operation of such websites and social accounts and that SecureDApp shall have no liability to you or any other person or entity for the use of third-party websites and social accounts. You are solely responsible for determining the extent to which you may use any content at any other websites and social accounts to which you link from the report.



ABOUT SECUREDAPP

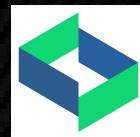
SecureDApp Auditor provides intelligent blockchain solutions. SecureDApp is developing an ecosystem that is seamless and responsive. Some of our services: Blockchain Security, Token Launchpad, NFT Marketplace, etc. SecureDApp's mission is to interconnect multiple services like Blockchain Security, DeFi, Gaming, and Marketplace under one ecosystem that is seamless, multi-chain compatible, scalable, secure, fast, responsive, and easy to use.

SecureDApp is built by a decentralised team of UI experts, contributors, engineers, and enthusiasts from all over the world. Our team currently consists of 6+ core team members and 10+ casual contributors. SecureDApp provides manual, static, and automatic smart contract analysis to ensure that the project is checked against known attacks and potential vulnerabilities.

To learn more, visit : <https://securedapp.in/>

To view our audit portfolio, visit : [github.securedapp.in](https://github.com/securedapp)

To book an audit, message : [securedapp.telegram](https://t.me/securedapp)



Securedapp.in



[Securedapp_Linkedin](https://www.linkedin.com/company/securedapp)



[Securedapp_Telegram](https://www.telegram.com/securedapp)